

CSC490 Assignment 5: Testing Your Project

Sohee Goo / 1008065479 Akshaya Deepak Ramachandran / 1008806810
Maryam Taj / 1008990362 Kashish Mittal / 1009115315

Contents

1	Part 1	2
1.1	Experiment Set Up	2
1.2	Results and Analysis	2
1.3	Modifying Functions based on these Results	3
2	Part 2 and 3: Test Coverage	5
2.1	Backend Testing	5
2.1.1	Setup	5
2.1.2	Test Suite Structure	5
2.1.3	Backend Coverage Results	6
2.2	Frontend Testing	6
2.2.1	Unit and Component Tests (Vitest)	6
2.2.2	Frontend Coverage Results	7
2.2.3	End-to-End Tests (Playwright)	7
2.3	CI/CD Workflows	8
2.4	Coverage Summary	8
3	Part 4	8
3.1	Module Picked: inference_service.py	8
3.2	Tests Added	9
3.3	Coverage Results	11
4	Part 5	11

1 Part 1

1.1 Experiment Set Up

Rather than profiling isolated functions directly, we designed a set of representative profiling scenarios that simulate realistic system workloads. This approach ensures that the identified functions are not only computationally expensive in isolation, but also significant in real-world usage.

Each scenario was selected to exercise a distinct subsystem of the application:

Authentication and Cryptography (`auth_crypto_burst`)

This scenario repeatedly performs password hashing/verification and JWT encoding/decoding. These operations are central to user authentication and are known to be computationally intensive, making them strong candidates for profiling.

Image Preprocessing (`preprocess_small_image`, `preprocess_high_resolution`)

These scenarios simulate batch image processing at different resolutions. Image preprocessing is a core part of the data pipeline and likely to dominate runtime due to resizing, patch generation, and encoding operations.

Inference Pipeline (`inference_patch_pipeline_full`)

This scenario represents a full end-to-end workflow, including preprocessing, storage interactions, and metadata handling. It was chosen to capture interactions between multiple components and identify bottlenecks in integrated workflows.

Dataset Splitting (`split_stratified_large_n`)

This scenario evaluates the performance of dataset partitioning logic at scale. Since data splitting is performed on large collections, inefficiencies can significantly impact preprocessing time.

Storage and Metadata Operations (`s3_inference_metadata_burst`)

This scenario simulates repeated storage and metadata operations. Although external I/O is mocked, it allows evaluation of application-level overhead in handling persistence logic.

To improve the reliability of the profiling results, each scenario was executed multiple times (5 runs per scenario) after an initial warm-up phase. This ensured that cold-start effects (e.g., module imports and library initialization) were excluded from measurement. Additionally, external dependencies (e.g., AWS services) were mocked to reduce variability and focus on CPU-bound performance.

Profiling was conducted using Python’s `cProfile` module, and results were analyzed based on cumulative execution time. From these results, the most time-consuming functions across all scenarios were identified and selected as the five functions for further investigation and optimization.

1.2 Results and Analysis

Scenario	Mean Time (s)	Function Calls	Dominant Cost
<code>auth_crypto_burst</code>	12.30 ± 0.09	6,185	<code>bcrypt.checkpw</code>
<code>preprocess_high_resolution</code>	5.78 ± 0.07	775,267	<code>ImageCore.resize</code> + JPEG encode
<code>s3_inference_metadata_burst</code>	0.93 ± 0.09	1,119,741	Botocore/moto SDK operations
<code>preprocess_small_image</code>	0.74 ± 0.01	112,867	<code>ImageCore.resize</code> + JPEG encode
<code>inference_patch_pipeline_full</code>	0.34 ± 0.07	331,764	Moto setup + botocore + preprocess
<code>split_stratified_large_n</code>	0.041 ± 0.002	69,841	<code>_stable_int</code> (SHA-256 hashing)

Table 1: Profiling scenario summary aggregated over 5 runs (warm-up excluded). Values are reported as mean $\pm$ standard deviation. Low variability is observed for CPU-bound workloads (e.g., authentication and preprocessing), while higher variability appears in scenarios involving mocked AWS services due to internal SDK and I/O overhead.

Note that across all repeated runs, the ranking of the most time-consuming functions remained consistent. For example:

- `bcrypt.checkpw` consistently dominated the authentication scenario (~12 s total runtime)
- `ImagingCore.resize` remained the primary cost in all preprocessing scenarios
- `_stable_int` consistently accounted for over 50% of dataset splitting time

This stability indicates that the identified hotspots are not artifacts of measurement noise, but instead reflect true performance bottlenecks in the system.

Based on cumulative execution time and consistency across runs, the following five functions were identified as the primary performance hotspots:

1. `verify_password`

The `bcrypt.checkpw` function called within the `verify_password` function accounts for approximately 97% of execution time in the authentication scenario (~0.30 s per call). Its high cost is expected, as `bcrypt` is intentionally designed to be computationally expensive for security reasons.

2. `downsample_to_square` (Image Preprocessing)

`ImagingCore.resize`, which is invoked by `downsample_to_square`, accounts for approximately 63–68% of preprocessing time across both small and high-resolution scenarios. Its cost scales with both image size and number of patches, making it the most significant CPU bottleneck in the data pipeline.

3. `_encode_jpeg` (Image Preprocessing)

`ImagingEncoder.encode`, invoked through `_encode_jpeg`, is responsible for approximately 16–17% of preprocessing time. It performs JPEG compression with high-quality settings (`quality=95`, `optimize=True`). While less dominant than resizing, it represents a meaningful secondary cost that could be reduced through parameter tuning.

4. `_stable_int` (Dataset Splitting)

The `_stable_int` function, used in `split.py`, is called 8,400 times during the stratified split scenario and accounts for approximately 51% of that scenario's wall time (0.011 s tottime). Internally, the cost is dominated by hashing operations, specifically `openssl_sha256` (0.003 s) and `hexdigest` (0.005 s). While the absolute runtime is small (~40 ms total), the function scales as $\mathcal{O}(N \times k)$ with dataset size and would reach approximately 1.4 s for 100k images.

5. `save_image_metadata` / `upload_raw_image` / `create_inference_record` (Persistence Layer)

The functions `save_image_metadata`, `upload_raw_image`, and `create_inference_record` in `inference_service.py` are thin wrappers around individual boto3 SDK calls. In the `s3_inference_metadata_burst` scenario, 112 API calls are issued, each passing through the full `botocore` pipeline, including hook dispatch, request signing, endpoint resolution, serialization, and the `moto` stub. Each request costs approximately 2–5 ms.

The dominant internal cost arises from `botocore_stubber.process_request`, along with the event hook system (e.g., `hooks.emit`), which is invoked over 1,200 times across all requests. With 35 images triggering three separate SDK calls each, the aggregate SDK overhead reaches approximately 0.5–0.7 s per run. Since application-level overhead is negligible, performance improvements could be made by reducing the number of SDK calls, for example by batching operations.

1.3 Modifying Functions based on these Results

Based on the profiling analysis, each identified hotspot suggests a different type of optimization, ranging from architectural changes to parameter tuning and algorithmic improvements.

1. `verify_password` (Authentication)

The cost of `verify_password` is dominated by `bcrypt.checkpw`, which is intentionally computationally expensive for security reasons. As such, direct optimization of this function is not appropriate, as reducing the cost factor would weaken password security.

Instead, improvements should be made at the system level. In an asynchronous web framework (e.g., FastAPI), this function should be executed in a background thread using `asyncio.to_thread()` or a thread pool executor. This prevents blocking the event loop and improves concurrency under high request load. Additionally, rate limiting authentication attempts can reduce the frequency of expensive calls.

2. `downsample_to_square` (Image Preprocessing)

The function currently uses bicubic interpolation:

```
return image.resize((size, size), resample=Image.Resampling.BICUBIC)
```

Since `ImageCore.resize` is the dominant cost in preprocessing, one possible optimization is to switch to a faster resampling method such as `BILINEAR`:

```
return image.resize((size, size), resample=Image.Resampling.BILINEAR)
```

This would reduce computation time at the expense of slightly lower image quality. For inference pipelines (as opposed to training or archival use), this trade-off is often acceptable. Alternatively, libraries such as `pillow-simd` could be used as a drop-in replacement to accelerate resizing.

3. `_encode_jpeg` (Image Preprocessing)

The current implementation uses high-quality and optimization settings:

```
img.save(buf, format="JPEG", quality=95, optimize=True)
```

Since JPEG encoding accounts for a significant portion of preprocessing time, performance can be improved by adjusting these parameters. For example:

```
img.save(buf, format="JPEG", quality=85, optimize=False)
```

Lowering the quality and disabling optimization reduces CPU overhead by avoiding additional compression passes. For small patches (e.g., 224×224), this typically has minimal impact on downstream inference accuracy while providing measurable performance gains.

4. `_stable_int` (Dataset Splitting)

The current implementation uses `hexdigest()` followed by string slicing and conversion:

```
h = hashlib.sha256(msg).hexdigest()
return int(h[:16], 16)
```

This introduces unnecessary intermediate string allocations. A more efficient approach is to operate directly on the binary digest:

```
h = hashlib.sha256(msg).digest()
return int.from_bytes(h[:8], "big")
```

This avoids hex string conversion and reduces overhead while preserving deterministic behavior. Although the absolute runtime is small, this optimization becomes more relevant at larger dataset sizes due to the function's $\mathcal{O}(N \times k)$ scaling.

5. `save_image_metadata` / `upload_raw_image` / `create_inference_record` (Persistence Layer)

These functions each perform individual boto3 SDK calls (e.g., `put_object`, `put_item`). Since each call incurs significant overhead from the `botocore` pipeline, performance can be improved by reducing the number of API calls.

One approach is to batch DynamoDB writes using `batch_write_item`, which allows multiple records to be written in a single request. For example, image metadata and inference records could be accumulated and written in batches rather than individually.

Additionally, if appropriate, uploads and metadata writes could be parallelized using asynchronous or concurrent execution to improve throughput. These changes would reduce per-request overhead and improve scalability in high-throughput scenarios.

2 Part 2 and 3: Test Coverage

NOTE: All the coverage information is up to date as of March 27th 11pm. There may be code changes after that point in order to finish the project, which might affect test coverage. Please refer to the repository at this timestamp when evaluating.

Please refer to `tests/` for all backend test files, `tests/README.md` for detailed backend testing documentation, and `src/apps/frontend/tests/` for frontend unit tests. End-to-end tests live in `src/apps/frontend/e2e/`.

The project contains **958 tests** across **69 test files**, spanning three testing layers: backend unit/integration tests (pytest), frontend unit/component tests (Vitest), and frontend end-to-end tests (Playwright). Both the backend and frontend achieve **100% statement coverage**.

2.1 Backend Testing

2.1.1 Setup

We use pytest as our test framework with pytest-cov for coverage reporting, moto for mocking AWS services (S3, DynamoDB, STS), httpx for async FastAPI integration tests, and Locust for production load testing. All test dependencies are listed in `requirements.txt` and pytest configuration lives in `pytest.ini`.

Coverage is configured via `.coveragerc` at the project root. We use the `omit` directive to exclude the five Modal GPU training modules (`train.py`, `evaluate.py`, `inference.py`, `dataset.py`, `model.py`) from coverage measurement because they require PyTorch and CUDA, which are not installed in the CI environment. These modules have their own dedicated test files (`test_model.py`, `test_train.py`, `test_evaluate.py`, `test_dataset.py`, `test_inference_modal.py`) that verify their logic on CPU with mocked datasets and checkpoints. Every other line of Python in `src/` is covered.

2.1.2 Test Suite Structure

The backend test suite contains **645 tests** across **33 test files**, organized by the module they cover. The shared test infrastructure lives in `confest.py`, which provides:

- **Mocked AWS:** moto-backed S3 buckets (2) and DynamoDB tables (5 with Global Secondary Indexes) that mirror the production Terraform schema — tests actually create items, query GSIs, upload objects, and generate presigned URLs rather than just mocking return values
- **FastAPI test client:** `httpx AsyncClient` wired to the app via `ASGITransport`, sending real HTTP requests through the full middleware stack
- **Auth helpers:** a `create_test_user()` factory that inserts users with bcrypt-hashed passwords, and pre-built JWT `auth_headers`
- **Image fixtures:** minimal valid JPEG bytes for upload tests

Unit Tests. Verify individual functions in isolation — password hashing and verification, JWT token creation/decoding/expiration, S3 URI parsing, environment variable validation, DynamoDB pagination helpers, status enums, deterministic data splitting (SHA-256 hash stability, stratification, k-fold reproducibility), image grid computation, and all data contract validators (field length limits, score clamping to 0.0–1.0, prediction validation for `{-1, 0, 1}`, filename sanitization including path traversal and null byte injection, schema `__post_init__` normalization for invalid labels, negative dimensions, and float-to-int conversion).

Service Tests. Cover the inference pipeline business logic (S3 upload with content/content-type verification, DynamoDB metadata writes with all fields validated, Modal inference calls, Bedrock RAG queries, presigned URL generation), user CRUD operations (create, get by ID/email, update profile, password changes, duplicate detection), and the data processing driver functions. Assertions verify exact field values, types, and timestamps.

Route Integration Tests. Use `httpx's AsyncClient` with `ASGITransport` to send real HTTP requests through the full FastAPI middleware stack (CORS, request logging, JWT authentication) against mocked AWS backends. These cover all API endpoints: signup, login, profile management, password changes, inference submission (including oversized file rejection at 20 MB, invalid image detection, whitespace-only metadata rejection, XSS/SQL injection strings in metadata fields), inference history (list with cursor-based pagination, get, delete, delete-all, stats), training/evaluation job spawning, RAG queries (empty query validation, snippet truncation to 200 chars, multiple citation flattening), and ECS task launching. They verify correct HTTP status codes for both success and error paths (400, 401, 404, 409, 422, 500) with specific error message assertions.

Load and Stress Tests. Use concurrent `asyncio.gather` to simulate real traffic patterns: 100 simultaneous health checks, 50 concurrent profile reads, 30 concurrent signups, burst invalid login attempts (verifying all return 401, not 500), mixed read/write workloads, and a throughput test confirming >100 requests/second. A separate Locust configuration (`tests/locustfile.py`) defines 4 user classes with weighted task distributions for production load testing against a live deployment.

Edge Case and Coverage Gap Tests. `test_coverage_gaps.py` specifically targets error-handling branches that are hard to reach in normal testing: DynamoDB `ClientError` propagation, presigned URL generation failures during inference history listing, malformed prediction values in database records, user records vanishing between update and re-fetch (concurrent delete / eventual consistency), generic exceptions during the inference pipeline prep phase, missing environment variables mid-request, the data pipeline driver continuing after a single image processing failure, MET pipeline progress logging at 10,000 records, and the Wikidata pipeline's `main()` orchestrator with both successful and empty query results.

2.1.3 Backend Coverage Results

645 passed, 0 failed

TOTAL: 1837 statements, 0 uncovered | 100% coverage

All Python modules under `src/` are measured except the five Modal GPU modules which are omitted via `.coveragerc`'s `omit` directive. Within measured files, we also exclude lines that cannot execute outside their runtime environment: `if __name__ == "__main__"` blocks (CLI entry points), Modal decorator lines (`@app.function`, `@app.local_entrypoint`) which require the Modal cloud runtime, and Modal remote invocations (`.remote()`, `.spawn()`) which require an active Modal deployment. Bash deployment scripts (`scripts/*.sh`) are not measurable by Python coverage tools and are tested manually during deployment.

2.2 Frontend Testing

The React frontend has a two-layer testing strategy: **Vitest unit/component tests** that verify every module in isolation with 100% statement coverage, and **Playwright end-to-end tests** that exercise the full application through a real browser.

2.2.1 Unit and Component Tests (Vitest)

We use Vitest with `@testing-library/react` for component rendering, `jsdom` as the DOM environment, and `v8` as the coverage provider. The test setup (`src/test/setup.ts`) mocks browser APIs unavailable in `jsdom` (`ResizeObserver`, `crypto.subtle`, pointer capture, canvas methods). Coverage is configured in `vite.config.ts` to include all source files under `src/app/` while excluding the `components/ui/` directory (third-party shadcn/ui components).

The frontend unit test suite contains **247 tests** across **29 test files**:

API Layer Tests (4 files, 25 tests). Test the HTTP client abstraction (`client.ts`), the inference analysis pipeline (`analysis.ts`), the inference history API (`inferencesApi.ts`), and the backend URL configuration (`backendApi.ts`). These verify correct request construction, JWT token attachment, mock-mode delay simulation, image dimension extraction via `FileReader`, error propagation, and the dual-mode architecture (mock mode vs. real API backend).

Component Tests (9 files, 62 tests). Cover all custom React components:

- **Header** (14 tests): authenticated vs. unauthenticated nav, dropdown menu, logout, custom auth links, loading state
- **ErrorBoundary** (11 tests): error catching and display, development vs. production mode, recovery via retry, null/undefined error handling
- **PatchOverlay** (9 tests): canvas heatmap drawing, tooltip positioning on mouse hover, overlay toggle, opacity slider, missing image placeholder, null `getContext` fallback
- **ProtectedRoute** (8 tests): redirect to login when unauthenticated, render child when authenticated, loading spinner during auth check
- **PageHeader, BrushDivider, RootLayout** (5 tests): rendering with props, action slots, layout structure

Context Tests (1 file, 25 tests). Exhaustively test the `AuthContext` provider in both mock mode and API backend mode: signup, login, logout, profile update, password change, legacy user migration from plaintext to hashed passwords, corrupted `localStorage` recovery, duplicate email rejection, JWT token persistence, API error handling, and the `useAuth` hook guard.

Page Tests (7 files, 110 tests). Each page is rendered inside `MemoryRouter` with `AuthProvider` and real route configuration:

- **HistoryPage** (30 tests): empty state, history from `localStorage`, image thumbnails, search filtering by artist/artwork/filename, score/prediction filtering (authentic, forged, uncertain, failed), all four sort modes, view details navigation, delete single item, clear all with confirmation dialog, API backend mode (list, paginate, delete, clear all, error handling for each), loading spinner
- **ProfilePage** (17 tests): form pre-fill, save button enable/disable, username and email validation, successful profile update, profile update error, password change validation (empty, too short, mismatch), successful password change, API stats display, corrupt `localStorage` stats
- **UploadPage** (11 tests): file extension validation, MIME type validation, size limit enforcement, drag-and-drop, file preview and removal, form validation (missing file, missing fields), successful submission with mock pipeline, API backend mode (no local history persistence), analyze error display
- **HomePage** (9 tests): hero content, navigation links, authenticated vs. unauthenticated state
- **ResultsPage** (6 tests): redirect when no stored result, invalid JSON handling, score display, failed inference dash, heatmap toggle, share/download actions
- **LoginPage** (4 tests): wrong password error, error clearing on input, password visibility toggle, successful login redirect
- **SignUpPage** (4 tests): successful signup with redirect after timeout, error clearing on input, duplicate email error

Library Tests (5 files, 41 tests). Test pure utility modules: analysis display helpers (verdict text, badge selection, score formatting, edge cases for all prediction/status combinations), PDF report generation (pagination across pages, image embedding, JPEG/PNG format detection, empty explanation fallback), upload size limits, environment detection, and art asset path resolution.

Type and Route Tests (2 files, 10 tests). Verify the TypeScript type guards (`getErrorMessage` for `Error`, `string`, and `unknown` types) and the React Router route configuration (all 7 routes resolve to the correct page components, protected routes wrap with `ProtectedRoute`).

2.2.2 Frontend Coverage Results

247 passed, 0 failed

All files: 100% Statements | 92.5% Branches | 96.8% Functions | 100% Lines

Coverage thresholds are enforced in `vite.config.ts` at 100% for lines and statements. The remaining branch gaps are JSX ternary expressions for conditional rendering and `hasApiBackend()` mode switches that are exercised behaviorally but not counted by v8's branch instrumentation.

2.2.3 End-to-End Tests (Playwright)

Playwright runs **66 test cases** across **7 spec files** against a full Vite production build served on `127.0.0.1:4173`. Tests use Desktop Chrome and exercise the complete user journey through a real browser:

- **auth.spec.ts** (14 tests): signup with unique credentials, duplicate email rejection, username minimum length validation, password minimum length validation, password visibility toggle, link to login, successful login, wrong password error, unknown email error, link to signup, already-authenticated redirect
- **navigation.spec.ts** (10 tests): home page hero and branding, “Get started” and “Log in” links, authenticated user navigation (Upload, History links), footer content, unknown route handling
- **upload.spec.ts** (9 tests): upload form display, analyze button disabled state, file preview rendering, file removal, file size validation (>20 MB error), full analysis pipeline from upload to results display (score, verdict, explanation, heatmap toggle, “analyze another artwork” link)
- **profile.spec.ts** (10 tests): user info pre-filled, save button disabled when unchanged, username validation, password change validation (mismatch, too short), account statistics display
- **history.spec.ts** (8 tests): empty state, analysis card display, view details navigation, delete single analysis, search by artist and artwork name, clear all with confirmation, analysis count

- **fullstack.spec.ts** (12 tests): real backend integration tests (skipped when `VITE_API_URL` is not set) covering signup, login, upload, results, history, profile, and logout against the live API with JWT authentication
- **smoke.spec.ts** (3 tests): home page renders, full end-to-end journey (signup → upload → analyze → results → history)

The Playwright configuration (`playwright.config.mjs`) automatically builds and serves the frontend via `vite preview` before running tests, uses 2 retries in CI with the GitHub reporter, and captures traces on first retry for debugging failures.

2.3 CI/CD Workflows

Two GitHub Actions workflows automate coverage reporting, both triggered on every push to `main` and every pull request:

Backend Coverage (`.github/workflows/test-coverage.yml`). Sets up Python 3.11, installs dependencies, exports AWS credential stubs with `ENVIRONMENT=dev` so moto mocks are used, runs the full pytest suite with `--cov-fail-under=100`, generates a `coverage.svg` badge using `coverage-badge` (committed to the repo on main branch pushes), posts a per-file coverage summary comment on pull requests via `orgoro/coverage`, and uploads the HTML coverage report as a downloadable artifact.

Frontend Coverage (`.github/workflows/frontend-coverage.yml`). Sets up Node.js 20, installs dependencies with `npm ci`, runs `vitest run --coverage`, generates a `frontend-coverage.svg` badge using `tj-actions/coverage-badge-js` (committed to the repo on main branch pushes), posts a coverage comment on pull requests, and uploads the coverage report as an artifact.

Both badges are displayed at the top of the README so reviewers can see coverage at a glance. The badges update dynamically on every push to main and they reflect actual test coverage results, not hardcoded values.

2.4 Coverage Summary

Layer	Tests	Files	Coverage
Backend (pytest)	645	33	100% (1,837 stmts)
Frontend unit (Vitest)	247	29	100% (stmts/lines)
Frontend e2e (Playwright)	66	7	All 7 routes
Total	958	69	100% backend + frontend

Table 2: Test suite summary across all layers.

The combination of 100% unit coverage on both backend and frontend, comprehensive API integration tests with real HTTP requests through the full middleware stack, and end-to-end browser tests covering every user-facing route provides high confidence that the system behaves correctly at every level of abstraction, from individual utility functions through service-layer business logic to the full user journey in a real browser.

3 Part 4

3.1 Module Picked: `inference_service.py`

We consider a module complex if it is an integration-heavy, business-critical service layer that coordinates multiple external systems and must maintain strict data and state consistency. Under this definition, `inference_service.py` is one of the most complex modules in the codebase. It orchestrates the full inference pipeline, including image upload (S3), metadata persistence (DynamoDB), patch generation, model inference (Modal), optional explanation generation (RAG pipeline), and final result handling (status updates, score normalization, and presigned URLs). This introduces multiple sources of failure, including misconfiguration (missing environment variables), partial failures (e.g., DynamoDB updates failing mid-batch), and external service errors. Although other modules in the codebase may have higher cyclomatic complexity scores (e.g., training or dataset processing scripts), those primarily reflect algorithmic complexity within isolated functions. In contrast, `inference_service.py` exhibits higher integration complexity and operational risk, as errors in this module directly affect user-facing inference results and system reliability.

3.2 Tests Added

The full unit test suite along with an explanation of why each test was added can be found at: Test Suite for `inference_service.py`.

The following table provides a mapping between each function in `inference_service.py` and its associated unit tests, along with a brief description of the behavior being validated.

Module Function	Test Function	Description
upload_raw_image	test_uploads_to_s3_and_returns_uri	Verifies successful upload and correct S3 URI format
upload_raw_image	test_missing_bucket_raises	Ensures error is raised when required environment variable is missing
upload_raw_image	test_custom_prefix	Confirms custom S3 prefix is applied correctly
upload_raw_image	test_default_prefix_is_inference	Validates default prefix behavior when unset
save_image_metadata	test_saves_all_fields_to_dynamodb	Checks all metadata fields are correctly stored
save_image_metadata	test_negative_dimensions_clamped_to_zero	Ensures invalid dimensions are sanitized
save_image_metadata	test_save_metadata_missing_table_env	Verifies failure when DynamoDB table is undefined
save_image_metadata	test_save_metadata_truncates_long_strings	Confirms long inputs are truncated to limits
create_inference_record	test_creates_record_with_all_initial_fields	Validates correct initialization of inference record
create_inference_record	test_custom_ttl_via_env_var	Ensures TTL is computed correctly from environment variable
create_inference_record	test_title_mirrors_artwork_name	Confirms title duplication and truncation logic
create_inference_record	test_create_record_negative_file_size	Ensures invalid file sizes are handled safely
create_and_upload_patches	test_creates_patches	Verifies patch generation and structure
create_and_upload_patches	test_patch_ddb_items_have_all_fields	Ensures DynamoDB patch entries contain required fields
create_and_upload_patches	test_create_patches_missing_bucket_env	Checks failure on missing configuration
create_and_upload_patches	test_custom_processed_prefix	Confirms prefix configuration for processed images
run_modal_inference	test_raises_on_failure	Ensures controlled error when external service fails
run_modal_inference	test_returns_result_on_success	Validates successful inference response handling
run_modal_inference	test_modal_correct_app_and_args	Confirms correct Modal function invocation parameters
save_patch_predictions	test_successful_patch_prediction_update	Verifies correct storage of predictions and scores
save_patch_predictions	test_multiple_patches_updated	Ensures batch updates work correctly
save_patch_predictions	test_empty_patch_list_is_noop	Confirms safe handling of empty input
save_patch_predictions	test_continues_after_first_patch_ddb_failure	Ensures resilience to partial failures
query_rag_explanation	test_returns_none_when_no_kb	Ensures RAG is skipped when not configured
query_rag_explanation	test_returns_none_on_pipeline_error	Ensures None is returned when the pipeline fails
query_rag_explanation	test_empty_pipeline_response_returns_none	Ensures None is returned when pipeline response is empty
query_rag_explanation	test_success_returns_pipeline_text	Confirms successful explanation generation
mark_inference_failed	test_updates_status	Verifies status update to failed
mark_inference_failed	test_mark_failed_truncates_long_error	Ensures long error messages are truncated
mark_inference_failed	test_mark_failed_logs_on_ddb_failure	Confirms failure is logged without crashing
finalize_inference	test_updates_with_results	Validates successful completion of inference record
finalize_inference	test_finalize_without_explanation	Handles optional explanation field correctly
finalize_inference	test_finalize_clamps_score_above_one	Ensures score upper bound enforcement
finalize_inference	test_finalize_clamps_score_below_zero	Ensures score lower bound enforcement
finalize_inference	test_finalize_rejects_invalid_prediction	Validates prediction value constraints
finalize_inference	test_finalize_clears_previous_error	Removes stale error messages on success
finalize_inference	test_finalize_logs_on_ddb_failure	Ensures resilience to database errors
generate_image_url	test_generates_url	Verifies successful presigned URL generation
generate_image_url	test_returns_none_on_error	Handles invalid inputs gracefully
generate_image_url	test_custom_presign_expiration	Ensures expiration configuration is respected
generate_image_url	test_presign_exception_returns_none	Handles presign failures safely
_now_ms	test_now_ms_returns_positive_int	Validates timestamp format and correctness

Table 3: Mapping of module functions to unit tests and their purposes

3.3 Coverage Results

To evaluate the depth of our unit tests within the `inference_service.py`, we used `pytest` with branch coverage enabled (`--cov-branch`). This ensures that not only individual lines of code are executed, but also all logical branches, including success paths, failure paths, exception handling, and conditional logic.

Results:

File	function	Statements				Branches			Total
		coverage	statements	missing	excluded	coverage	branches	partial	
src/apps/backend/services/inference_service.py	upload_raw_image	100%	6	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	save_image_metadata	100%	2	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	create_inference_record	100%	5	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	create_and_upload_patches	100%	8	0	0	100%	2	0	100%
src/apps/backend/services/inference_service.py	run_modal_inference	100%	14	0	1	100%	0	0	100%
src/apps/backend/services/inference_service.py	save_patch_predictions	100%	6	0	0	100%	2	0	100%
src/apps/backend/services/inference_service.py	query_rag_explanation	100%	21	0	0	100%	4	0	100%
src/apps/backend/services/inference_service.py	finalize_inference	100%	12	0	0	100%	2	0	100%
src/apps/backend/services/inference_service.py	mark_inference_failed	100%	5	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	generate_image_url	100%	6	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	_now_ms	100%	1	0	0	100%	0	0	100%
src/apps/backend/services/inference_service.py	(no function)	100%	28	0	0	100%	0	0	100%
Total		100%	114	0	1	100%	10	0	100%

coverage.py v7.13.5, created at 2026-03-27 10:45 -0400

Figure 1: HTML coverage report showing branch coverage and missing lines

The report confirms that both normal execution paths and edge-case branches are covered by the test suite.

4 Part 5

Note on Video Content: The following Google Drive link contains a large video file; please download it to watch. The first bug is demonstrated via a slide presentation, while the second is detailed through a Markdown file and a live demo—formats chosen to best suit each specific use case. While we address more than two issues, the bugs within each section are closely related, highlighting the interconnected challenges encountered while pursuing each development goal.

- Link to Video
- Link to Bug #2 Writeup